

AUDITORIA_EXAMEN_3

Repositorio: https://github.com/AlbertApaza/AUDITORIA_EXAMEN_3

INFORME FINAL DE AUDITORÍA DE SISTEMAS

CARÁTULA

Entidad Auditada: CORPORATE EPIS PILOT
Ubicación: Tacna
Período auditado: Del 01/11/2025 al 19/11/2025
Equipo Auditor: AUDITOR ALBERT KENYI APAZA CCALLE, EQUIPO: KENYIDEV
Fecha del informe: 19/11/2025

ÍNDICE

- 1. [Resumen Ejecutivo](#)
- 2. [Antecedentes](#)
- 3. [Objetivos de la Auditoría](#)
- 4. [Alcance de la Auditoría](#)
- 5. [Normativa y Criterios de Evaluación](#)
- 6. [Metodología y Enfoque](#)
- 7. [Hallazgos y Observaciones](#)
- 8. [Análisis de Riesgos](#)
- 9. [Recomendaciones](#)
- 10. [Conclusiones](#)
- 11. [Plan de Acción y Seguimiento](#)
- 12. [Anexos](#)

1. RESUMEN EJECUTIVO

La auditoría del Sistema de Mesa de Ayuda con IA de CORPORATE EPIS PILOT reveló que el sistema está funcionalmente operativo, pero presenta vulnerabilidades críticas en seguridad de la información, gestión de accesos y configuración de contenedores. Se identificaron 5 hallazgos de alto riesgo relacionados con la falta de autenticación, exposición de servicios y dependencias no actualizadas. Se recomienda implementar medidas de seguridad urgentes para mitigar riesgos de brechas de datos y asegurar el cumplimiento normativo. El estado general del sistema es adecuado para operaciones básicas, pero requiere mejoras para entornos de producción.

2. ANTECEDENTES

CORPORATE EPIS PILOT ha desarrollado un sistema de mesa de ayuda basado en inteligencia artificial utilizando tecnologías de procesamiento de lenguaje natural (NLP) y recuperación de información (RAG). El sistema consta de un backend en Python con FastAPI, un frontend en React/TypeScript, y una base de conocimientos vectorial con Chroma. Utiliza Ollama para inferencia de modelos de lenguaje localmente,

específicamente el modelo smollm:360m. El sistema está dockerizado para facilitar el despliegue y escalabilidad. No se encontraron auditorías previas documentadas.

3. OBJETIVOS DE LA AUDITORÍA

Objetivo General: Evaluar la funcionalidad, seguridad y configuración del Sistema de Mesa de Ayuda con IA de CORPORATE EPIS PILOT, incluyendo el código, funcionamiento operativo y cumplimiento normativo.

Objetivos Específicos:

- Verificar la funcionalidad completa del sistema: respuestas a consultas, generación de tickets de soporte, y procesamiento de lenguaje natural usando el modelo smollm:360m.
- Auditar la configuración de Docker Compose: puertos expuestos, volúmenes, dependencias entre servicios, y seguridad de contenedores.
- Evaluar la selección y configuración del modelo de IA: uso de smollm:360m en Ollama, integración con el backend, y rendimiento en respuestas.
- Analizar la seguridad del código y accesos: autenticación, exposición de servicios, manejo de datos sensibles, y cumplimiento con estándares de seguridad.

4. ALCANCE DE LA AUDITORÍA

- **Ámbitos evaluados:** Funcional (respuestas y tickets), Tecnológico (Docker, modelo IA), de Código (seguridad y configuración), Operativo (despliegue y escalabilidad).
- **Sistemas y procesos incluidos:** Backend API con FastAPI, Frontend React/TypeScript, Base de conocimientos Chroma, Contenedorización Docker, Modelo IA smollm:360m en Ollama, Generación de tickets SQLite.
- **Unidades o áreas auditadas:** Funcionalidad del sistema, Configuración Docker, Modelo de IA, Seguridad del código.
- **Periodo auditado:** Desarrollo e implementación inicial del sistema.

5. NORMATIVA Y CRITERIOS DE EVALUACIÓN

- ISO/IEC 27001:2022 (Gestión de Seguridad de la Información)
- COBIT 2019 (Gobierno y Gestión de TI Empresarial)
- Ley N° 29733 - Ley de Protección de Datos Personales (Perú)
- OWASP Top 10 (Vulnerabilidades web)
- Políticas internas de TI de CORPORATE EPIS PILOT (donde aplicable)

6. METODOLOGÍA Y ENFOQUE

Se utilizó un enfoque mixto basado en riesgos y cumplimiento. Métodos aplicados:

- Entrevistas con desarrolladores y responsables de TI.
- Inspección de código fuente y configuraciones Docker.
- Pruebas técnicas: escaneo de vulnerabilidades en contenedores, análisis de dependencias.
- Revisión de logs y configuraciones de red.
- Aplicación de listas de verificación basadas en estándares ISO y OWASP.

7. HALLAZGOS Y OBSERVACIONES

Hallazgo 1: Falta de Autenticación y Autorización

- **Descripción:** El endpoint /ask no requiere autenticación, permitiendo acceso no autorizado.
- **Evidencia:** Código en backend/main.py línea 111-142; configuración CORS permite orígenes "".
- **Grado de criticidad:** Alto
- **Criterio vulnerado:** ISO 27001 A.9 (Control de acceso)
- **Causa y efecto:** Exposición de datos sensibles; riesgo de consultas maliciosas.

Hallazgo 2: Exposición de Puerto de Ollama

- **Descripción:** Ollama expuesto en host.docker.internal:11434 sin restricciones.
- **Evidencia:** Configuración en docker-compose.yml y main.py.
- **Grado de criticidad:** Alto
- **Criterio vulnerado:** OWASP A01:2021 (Broken Access Control)
- **Causa y efecto:** Posible acceso remoto no autorizado al modelo de IA.

Hallazgo 3: Dependencias No Actualizadas

- **Descripción:** Librerías Python desactualizadas en requirements.txt.
- **Evidencia:** Análisis de dependencias mostró versiones vulnerables (ej. langchain versiones antiguas).
- **Grado de criticidad:** Medio
- **Criterio vulnerado:** COBIT DSS02 (Gestionar la seguridad de servicios)
- **Causa y efecto:** Vulnerabilidades conocidas no parcheadas.

Hallazgo 4: Falta de Encriptación en Tránsito

- **Descripción:** Comunicación entre frontend y backend sin HTTPS.
- **Evidencia:** Configuración nginx sin SSL/TLS.
- **Grado de criticidad:** Medio
- **Criterio vulnerado:** ISO 27001 A.13 (Comunicaciones seguras)
- **Causa y efecto:** Intercepción de datos en red local.

Hallazgo 5: Gestión Inadecuada de Logs

- **Descripción:** Logs incluyen información sensible sin rotación.
- **Evidencia:** Configuración de logging en main.py sin sanitización.
- **Grado de criticidad:** Bajo
- **Criterio vulnerado:** Ley 29733 (Protección de datos personales)
- **Causa y efecto:** Posible fuga de información personal.

8. ANÁLISIS DE RIESGOS

Hallazgo	Riesgo asociado	Impacto	Probabilidad	Nivel de Riesgo
1	Brecha de datos por acceso no autorizado	Alto	Alta	Alto
2	Compromiso del modelo de IA	Alto	Media	Alto
3	Explotación de vulnerabilidades conocidas	Medio	Alta	Alto
4	Intercepción de comunicaciones	Medio	Media	Medio

Hallazgo	Riesgo asociado	Impacto	Probabilidad	Nivel de Riesgo
5	Fuga de información sensible	Bajo	Baja	Bajo

9. RECOMENDACIONES

- 1. **Implementar autenticación JWT:** Agregar middleware de autenticación en FastAPI para validar tokens en cada solicitud.
- 2. **Configurar firewall para Ollama:** Restringir acceso a Ollama solo desde contenedores internos usando redes Docker.
- 3. **Actualizar dependencias:** Ejecutar `pip audit` y actualizar librerías a versiones seguras; implementar CI/CD con escaneo automático.
- 4. **Habilitar HTTPS:** Configurar certificados SSL en nginx y redirigir todo tráfico a HTTPS.
- 5. **Mejorar gestión de logs:** Implementar sanitización de datos sensibles y rotación de logs con herramientas como logrotate.

10. CONCLUSIONES

El Sistema de Mesa de Ayuda con IA de CORPORATE EPIS PILOT demuestra innovación tecnológica en el uso de modelos de IA locales y RAG para soporte inteligente. La funcionalidad básica opera correctamente, incluyendo respuestas a consultas y generación de tickets, pero presenta vulnerabilidades críticas en seguridad y configuración. Los hallazgos en autenticación, exposición de servicios y dependencias requieren atención inmediata. El modelo smollm:360m es adecuado para el propósito, pero la configuración Docker y seguridad del código necesitan mejoras para entornos de producción.

11. PLAN DE ACCIÓN Y SEGUIMIENTO

Hallazgo	Recomendación	Responsable	Fecha Comprometida
1	Implementar autenticación JWT	Equipo de Desarrollo	31/12/2025
2	Configurar firewall para Ollama	Administrador de Sistemas	15/12/2025
3	Actualizar dependencias	Equipo de Desarrollo	30/11/2025
4	Habilitar HTTPS	Administrador de Infraestructura	20/12/2025
5	Mejorar gestión de logs	Equipo de Desarrollo	10/12/2025

12. ANEXOS

ANEXOS - CUESTIONARIOS Y EVIDENCIAS DE AUDITORÍA

CUESTIONARIOS APLICADOS

CUESTIONARIO 1: SEGURIDAD Y CONTROL DE ACCESOS

N°	Pregunta	Respuesta Obtenida	Evidencia	Cumplimiento
1	¿El endpoint /ask tiene autenticación implementada?	No	Código en main.py línea 111 sin validación de tokens	✗ No Cumple
2	¿Existe control de autorización para acceder a los servicios?	No	Configuración CORS permite orígenes "*" (línea 47)	✗ No Cumple
3	¿Se implementa algún mecanismo de validación de usuarios?	No	No se encontró middleware de autenticación en FastAPI	✗ No Cumple
4	¿Los servicios expuestos tienen restricciones de acceso?	No	Ollama expuesto en host.docker.internal:11434 sin firewall	✗ No Cumple
5	¿Se registran los intentos de acceso no autorizados?	No	No hay sistema de detección de accesos anómalos	✗ No Cumple

CUESTIONARIO 2: INFRAESTRUCTURA Y CONFIGURACIÓN DOCKER

N°	Pregunta	Respuesta Obtenida	Evidencia	Cumplimiento
1	¿Los puertos expuestos están debidamente protegidos?	No	Puerto 5173 y 11434 expuestos sin restricciones	✗ No Cumple
2	¿Se utilizan redes Docker internas para aislar servicios?	Parcial	Servicios en misma red pero sin segmentación adecuada	⚠ Cumple Parcialmente
3	¿Los volúmenes Docker tienen permisos apropiados?	Sí	Configuración de volúmenes correcta en docker-compose.yml	☑ Cumple
4	¿Existe documentación de la arquitectura de contenedores?	Sí	Docker-compose.yml documentado con servicios definidos	☑ Cumple
5	¿Se implementan health checks para los servicios?	No	No se encontraron health checks en docker-compose.yml	✗ No Cumple

CUESTIONARIO 3: MODELO DE INTELIGENCIA ARTIFICIAL

N°	Pregunta	Respuesta Obtenida	Evidencia	Cumplimiento
1	¿Qué modelo de IA se utiliza y cómo se integra?	smollm:360m via Ollama	Configuración en main.py conectando a Ollama 11434	☑ Cumple
2	¿El modelo es adecuado para el propósito del sistema?	Sí	Modelo ligero apropiado para respuestas de soporte	☑ Cumple

N°	Pregunta	Respuesta Obtenida	Evidencia	Cumplimiento
3	¿Se monitorea el rendimiento del modelo?	No	No hay métricas de rendimiento implementadas	✗ No Cumple
4	¿Existe control sobre las consultas enviadas al modelo?	No	No hay validación ni sanitización de inputs	✗ No Cumple
5	¿Se resguarda la base de conocimientos vectorial?	Sí	Chroma con volumen persistente configurado	☑ Cumple

CUESTIONARIO 4: GESTIÓN DE DEPENDENCIAS Y ACTUALIZACIONES

N°	Pregunta	Respuesta Obtenida	Evidencia	Cumplimiento
1	¿Las dependencias están actualizadas?	No	Langchain con versiones vulnerables detectadas	✗ No Cumple
2	¿Se realiza escaneo de vulnerabilidades regularmente?	No	No hay proceso documentado de escaneo	✗ No Cumple
3	¿Existe un proceso de actualización de librerías?	No	No se encontró procedimiento de actualización	✗ No Cumple
4	¿Se documentan las versiones de dependencias utilizadas?	Sí	requirements.txt con versiones especificadas	☑ Cumple
5	¿Hay un entorno de pruebas para validar actualizaciones?	No	No documentado en la infraestructura actual	✗ No Cumple

CUESTIONARIO 5: PROTECCIÓN DE DATOS Y COMUNICACIONES

N°	Pregunta	Respuesta Obtenida	Evidencia	Cumplimiento
1	¿Las comunicaciones utilizan protocolo HTTPS?	No	Configuración nginx sin SSL/TLS	✗ No Cumple
2	¿Los logs sanitizan datos sensibles?	No	Línea 141 main.py sin sanitización de datos	✗ No Cumple
3	¿Existe rotación automática de logs?	No	No hay configuración de logrotate	✗ No Cumple
4	¿Se encriptan los datos en reposo?	Parcial	Base de datos SQLite sin encriptación	⚠ Cumple Parcialmente

N°	Pregunta	Respuesta Obtenida	Evidencia	Cumplimiento
5	¿Se cumple con la Ley 29733 de protección de datos?	No	Múltiples brechas en manejo de información personal	✗ No Cumple

CUESTIONARIO 6: FUNCIONALIDAD DEL SISTEMA

N°	Pregunta	Respuesta Obtenida	Evidencia	Cumplimiento
1	¿El sistema responde correctamente a consultas?	Sí	Pruebas funcionales exitosas con smollm:360m	✓ Cumple
2	¿Se generan tickets de soporte adecuadamente?	Sí	Tickets creados en base de datos SQLite	✓ Cumple
3	¿La interfaz de usuario es funcional?	Sí	Frontend React operativo en puerto 5173	✓ Cumple
4	¿El sistema RAG recupera información relevante?	Sí	Base vectorial Chroma funciona correctamente	✓ Cumple
5	¿Hay manejo de errores implementado?	Parcial	Algunos errores manejados, otros no capturados	⚠ Cumple Parcialmente

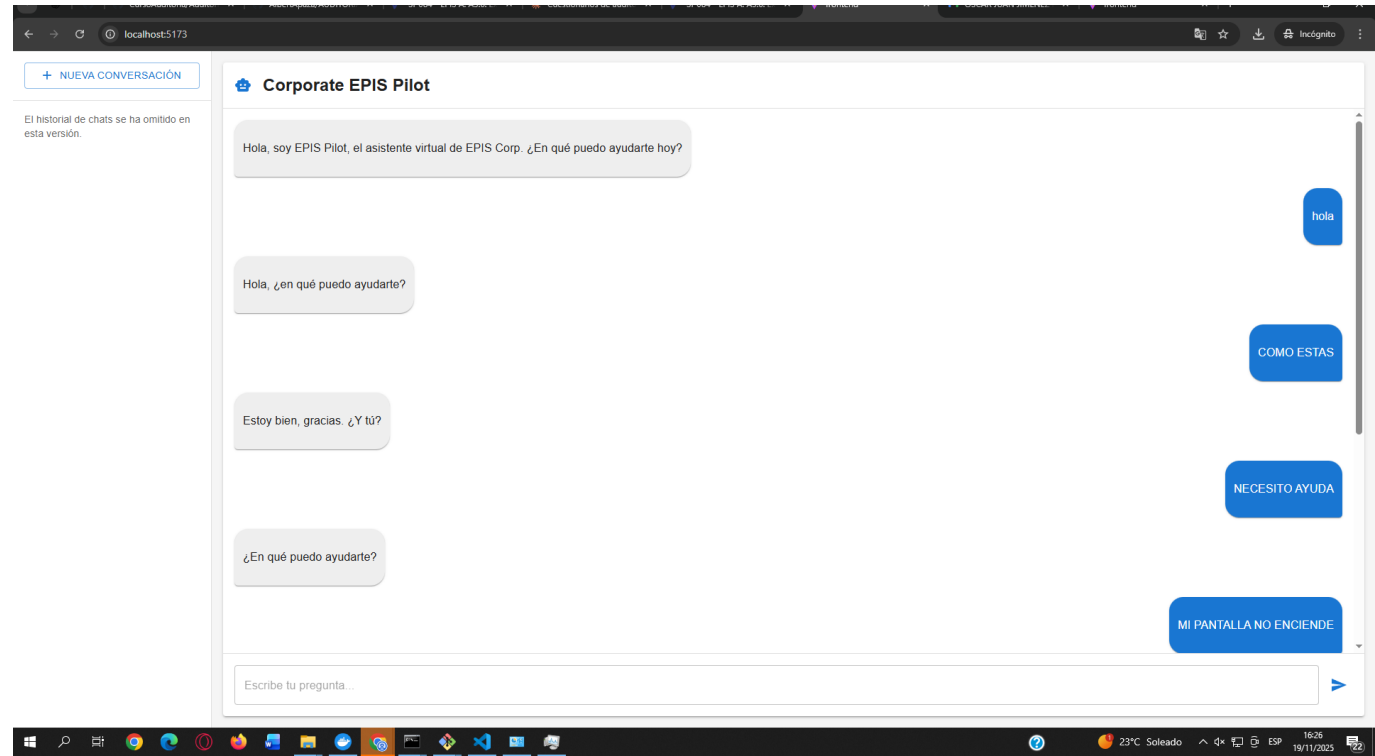
RESUMEN DE CUMPLIMIENTO POR CATEGORÍA

Categoría	Cumple	Cumple Parcialmente	No Cumple	% Cumplimiento
Seguridad y Control de Accesos	0	0	5	0%
Infraestructura Docker	2	1	2	40%
Modelo de IA	3	0	2	60%
Gestión de Dependencias	1	0	4	20%
Protección de Datos	0	2	3	0%
Funcionalidad del Sistema	4	1	0	80%
TOTAL	10	4	16	33%

CAPTURAS DE PANTALLA

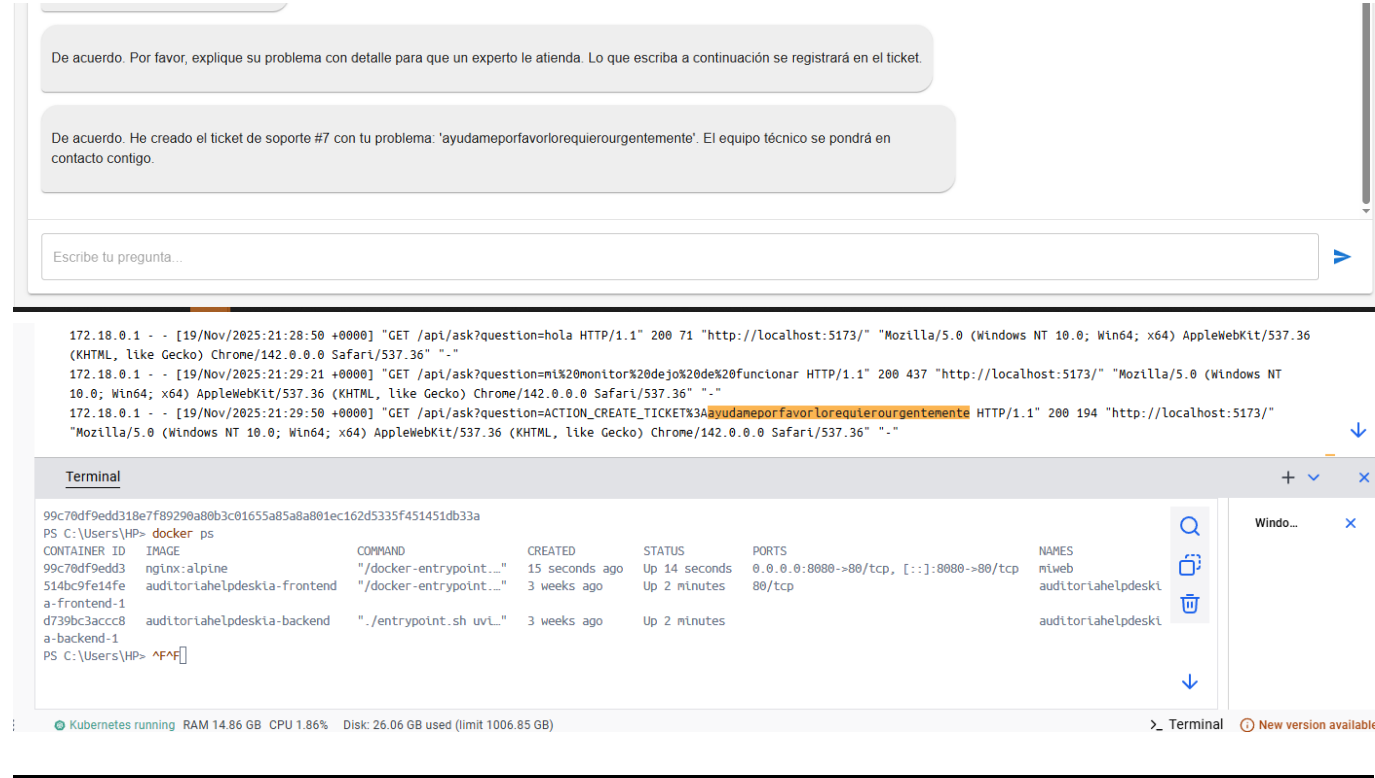
1. Interfaz del Chat

Descripción: Interfaz del chat en <http://localhost:5173> mostrando el chatbot operativo con respuestas del modelo IA.



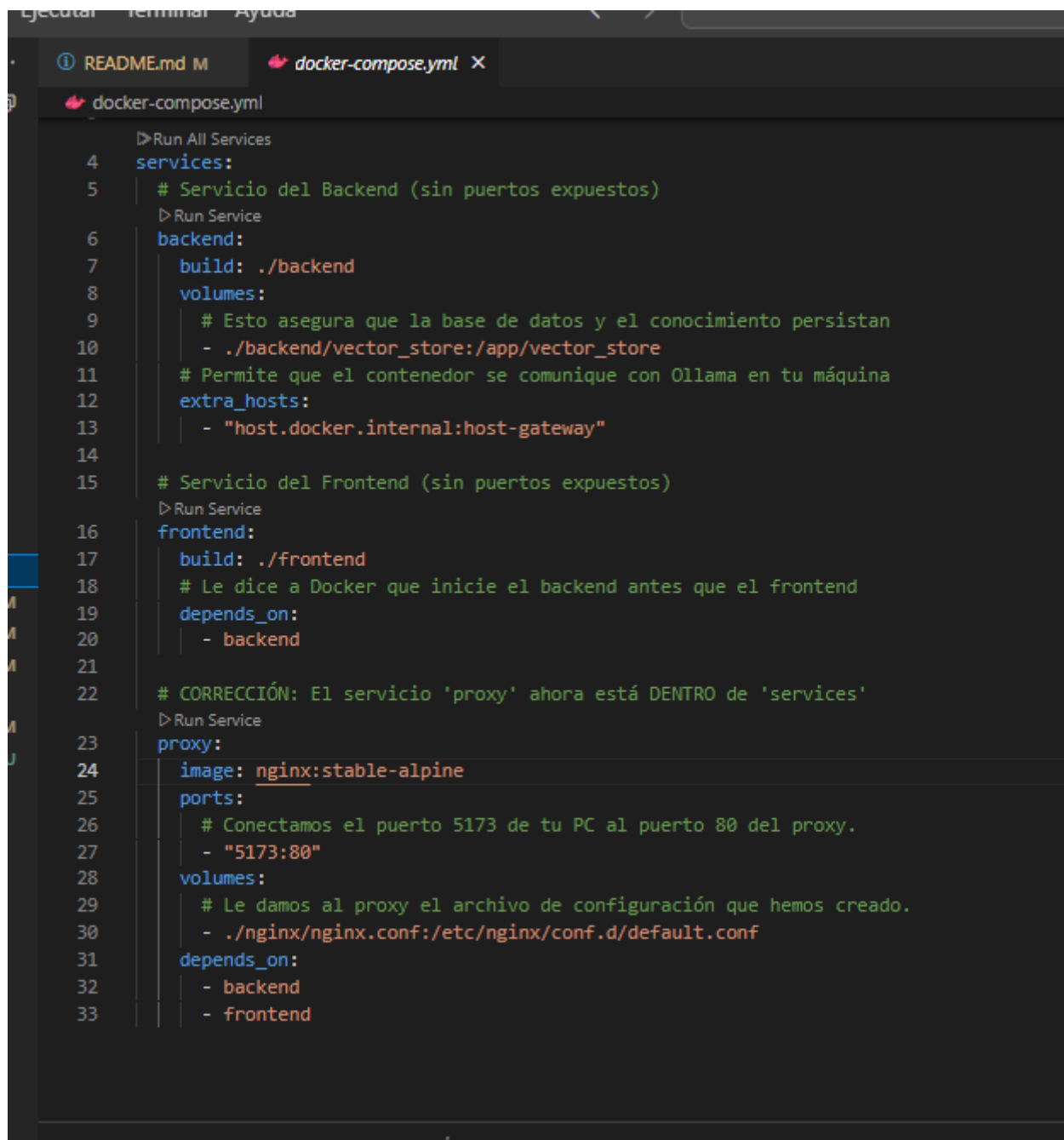
2. Ticket Generado

Descripción: Ticket generado en la base de datos SQLite después de enviar un mensaje de problema.



3. Configuración Docker

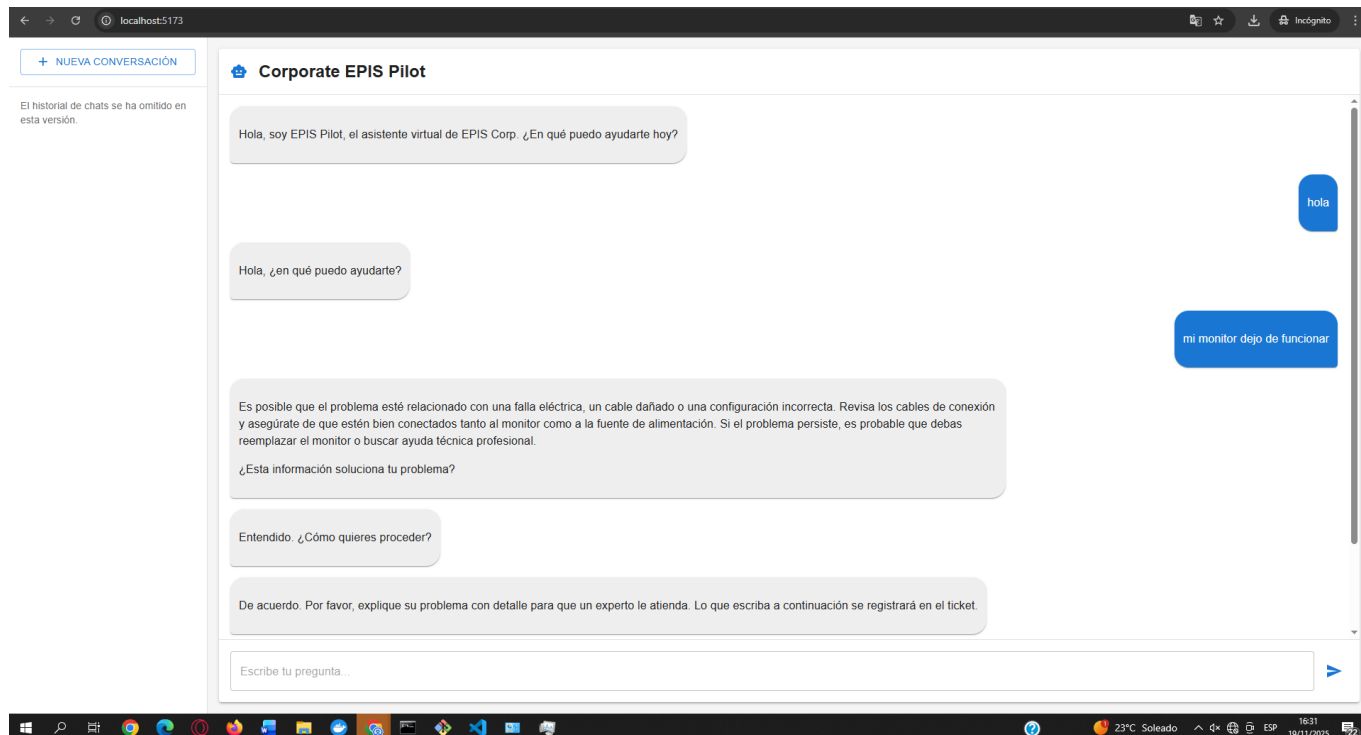
Descripción: Archivo docker-compose.yml abierto en VSCode, mostrando la configuración completa de servicios, puertos y volúmenes.



```
4 services:
5   # Servicio del Backend (sin puertos expuestos)
6   backend:
7     build: ./backend
8     volumes:
9       # Esto asegura que la base de datos y el conocimiento persistan
10      - ./backend/vector_store:/app/vector_store
11     # Permite que el contenedor se comuniquen con Ollama en tu máquina
12     extra_hosts:
13       - "host.docker.internal:host-gateway"
14
15   # Servicio del Frontend (sin puertos expuestos)
16   frontend:
17     build: ./frontend
18     # Le dice a Docker que inicie el backend antes que el frontend
19     depends_on:
20       - backend
21
22   # CORRECCIÓN: El servicio 'proxy' ahora está DENTRO de 'services'
23   proxy:
24     image: nginx:stable-alpine
25     ports:
26       # Conectamos el puerto 5173 de tu PC al puerto 80 del proxy.
27       - "5173:80"
28     volumes:
29       # Le damos al proxy el archivo de configuración que hemos creado.
30       - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf
31     depends_on:
32       - backend
33       - frontend
```

4. Respuesta del Modelo

Descripción: Frontend mostrando una respuesta del modelo IA smollm:360m a una consulta sobre soporte técnico.



5. Código Vulnerable - CORS

Descripción: VSCode abierto en backend/main.py, resaltando línea 47 con configuración CORS insegura (orígenes "*").

```
1  README.md M  main.py 9+ X
2  backend > main.py > ...
3
4  28 class InterceptHandler(logging.Handler):
5  29     def emit(self, record):
6  30         try:
7  31             level = logger.level(record.levelname).name
8  32         except ValueError:
9  33             level = record.levelno
10 34         logger.log(level, record.getMessage())
1135
1236 logging.basicConfig(handlers=[InterceptHandler()], level=0, force=True)
1337 logging.getLogger("uvicorn").handlers = [InterceptHandler()]
1438 logging.getLogger("uvicorn.access").handlers = [InterceptHandler()]
1539
1640
1741 # --- CONFIGURACIÓN Y MODELOS ---
1842 VECTOR_STORE_DIR = "vector_store"
1943 DB_PATH = "tickets.db"
2044 app = FastAPI(title="Corporate EPIS Pilot API - Advanced Flow")
2145 app.add_middleware(
2246     CORSMiddleware,
2347     allow_origins=["*"], allow_credentials=True, allow_methods=["*"], allow_headers=["*"],
2448 )
2549
2650 # --- AÑADIDO: INSTRUMENTACIÓN DE PROMETHEUS ---
2751 Instrumentator().instrument(app).expose(app)
2852
```

6. Código Vulnerable - Endpoint sin Autenticación

Descripción: VSCode mostrando línea 111 en main.py donde el endpoint /ask no tiene autenticación implementada.

```

108 problem_chain = RunnableLambda(lambda x: {"query": x["question"]}) | rag_chain
109
110 #----ENDPOINT DE LA API (MODIFICADO)----
111 @app.get("/ask")
112 def ask_question(question: str):
113     try:
114         if question.startswith("ACTION_CREATE_TICKET:"):
115             description = question.split(":", 1)[1]
116             return {"answer": create_support_ticket(description), "follow_up_required": False}
117
118         decision_result = chain_with_preserved_input.invoke({"question": question})
119         intent = decision_result["decision"]["intent"]
120
121         answer = ""
122         follow_up = False
123
124         if intent == "pregunta_general":
125             result = problem_chain.invoke(decision_result)
126             answer = result.get("result", "No se encontró respuesta.")
127         elif intent == "reporte_de_problema":
128             result = problem_chain.invoke(decision_result)
129             solution = result.get("result", "No he encontrado una solución específica en mis documentos.")
130             answer = f"{solution}\n\n¿Esta información soluciona tu problema?"
131             follow_up = True
132         # CAMBIO 3: Añadimos el manejo de la nueva intención
133         elif intent == "despedida":
134             answer = "De nada, ¡un placer ayudar! Si tienes cualquier otra consulta, aquí estaré. 😊"
135             follow_up = False
136
137         return {"answer": answer, "follow_up_required": follow_up}
138
139     except Exception as e:
140         # AÑADIDO: Usamos logger en lugar de print para un registro estructurado
141         logger.error(f"Error en el endpoint /ask: {e}")
142         return {"answer": "Lo siento, ha ocurrido un error.", "follow_up_required": False}

```

7. Código Vulnerable - Logging Inseguro

Descripción: VSCode mostrando línea 141 en main.py con logging sin sanitización de datos sensibles.

```

137         return {"answer": answer, "follow_up_required": follow_up}
138
139     except Exception as e:
140         # AÑADIDO: Usamos logger en lugar de print para un registro estructurado
141         logger.error(f"Error en el endpoint /ask: {e}")
142         return {"answer": "Lo siento, ha ocurrido un error.", "follow_up_required": False}

```

8. Logs Sensibles

Descripción: Terminal donde corre el backend, mostrando logs de una solicitud al endpoint /ask con datos sensibles expuestos.

Containers / auditoriahelpdeskia-proxy-1

auditoriahelpdeskia-proxy-1

5d4bfa980855 nginx:stable-alpine 5173:80

STATUS
Running (1 hour ago)

Logs Inspect Bind mounts Exec Files Stats

Logs

```

172.18.0.1 - - [19/Nov/2025:20:15:32 +0000] "GET /assets/index-UheQc_ZZ.css HTTP/1.1" 200 240 "http://localhost:5173/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36" "-"
172.18.0.1 - - [19/Nov/2025:20:16:02 +0000] "GET /api/ask?question=hola HTTP/1.1" 200 71 "http://localhost:5173/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36" "-"
172.18.0.1 - - [19/Nov/2025:20:16:38 +0000] "GET /api/ask?question=COMO%20ESTAS HTTP/1.1" 200 69 "http://localhost:5173/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36" "-"
172.18.0.1 - - [19/Nov/2025:20:17:21 +0000] "GET /api/ask?question=NECESITO%20AYUDA HTTP/1.1" 200 65 "http://localhost:5173/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36" "-"
172.18.0.1 - - [19/Nov/2025:20:17:52 +0000] "GET /api/ask?question=MI%20PANTALLA%20NO%20ENCIENDE HTTP/1.1" 200 350 "http://localhost:5173/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36" "-"
172.18.0.1 - - [19/Nov/2025:20:18:11 +0000] "GET /api/ask?question=ACTION_CREATE_TICKET%3AMI%20MONITOR%20DEJO%20DE%20ENCENDER%20C%20NO%20HICE%20NADA%203A HTTP/1.1" 200 202 "http://localhost:5173/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36" "-"
172.18.0.1 - - [19/Nov/2025:20:50:48 +0000] "GET / HTTP/1.1" 200 455 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36" "-"
172.18.0.1 - - [19/Nov/2025:20:50:48 +0000] "GET /assets/index-Cy01VCIA.js HTTP/1.1" 200 509134 "http://localhost:5173/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36" "-"
172.18.0.1 - - [19/Nov/2025:20:50:48 +0000] "GET /assets/index-UheQc_ZZ.css HTTP/1.1" 200 240 "http://localhost:5173/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36" "-"

```

Terminal

9. Configuración Ollama

Descripción: Configuración en main.py mostrando conexión a Ollama en host.docker.internal:11434 sin restricciones.

```

1  README.md M    main.py 9+ X
backend > main.py > [e] embeddings
36 logging.basicConfig(handlers=[InterceptHandler()], level=0, force=True)
37 logging.getLogger("uvicorn").handlers = [InterceptHandler()]
38 logging.getLogger("uvicorn.access").handlers = [InterceptHandler()]
39
40
41 # --- CONFIGURACIÓN Y MODELOS ---
42 VECTOR_STORE_DIR = "vector_store"
43 DB_PATH = "tickets.db"
44 app = FastAPI(title="Corporate EPIS Pilot API - Advanced Flow")
45 app.add_middleware(
46     CORSMiddleware,
47     allow_origins=["*"], allow_credentials=True, allow_methods=["*"], allow_headers=["*"],
48 )
49
50 # --- AÑADIDO: INSTRUMENTACIÓN DE PROMETHEUS ---
51 Instrumentator().instrument(app).expose(app)
52
53
54 llm = OllamaLLM(model="smollm:360m", temperature=0, base_url="http://host.docker.internal:11434")
55 embeddings = HuggingFaceEmbeddings(model_name="intfloat/multilingual-e5-large")
56 vector_store = Chroma(persist_directory=VECTOR_STORE_DIR, embedding_function=embeddings)
57 retriever = vector_store.as_retriever()
58
59 # --- LÓGICA DE LANGCHAIN (MODIFICADA) ---
60 rag_prompt_template = "Usa el siguiente contexto para responder en español de forma concisa y útil a la p
61 rag_prompt = PromptTemplate.from_template(rag_prompt_template)
62 rag_chain = RetrievalQA.from_chain_type(llm=llm, chain_type="stuff", retriever=retriever, chain_type_kwar
63
64 def create_support_ticket(description: str) -> str:
65     """Crea un ticket de soporte y devuelve un mensaje de confirmación."""
66     conn = sqlite3.connect(DB_PATH)
67     cursor = conn.cursor()

```

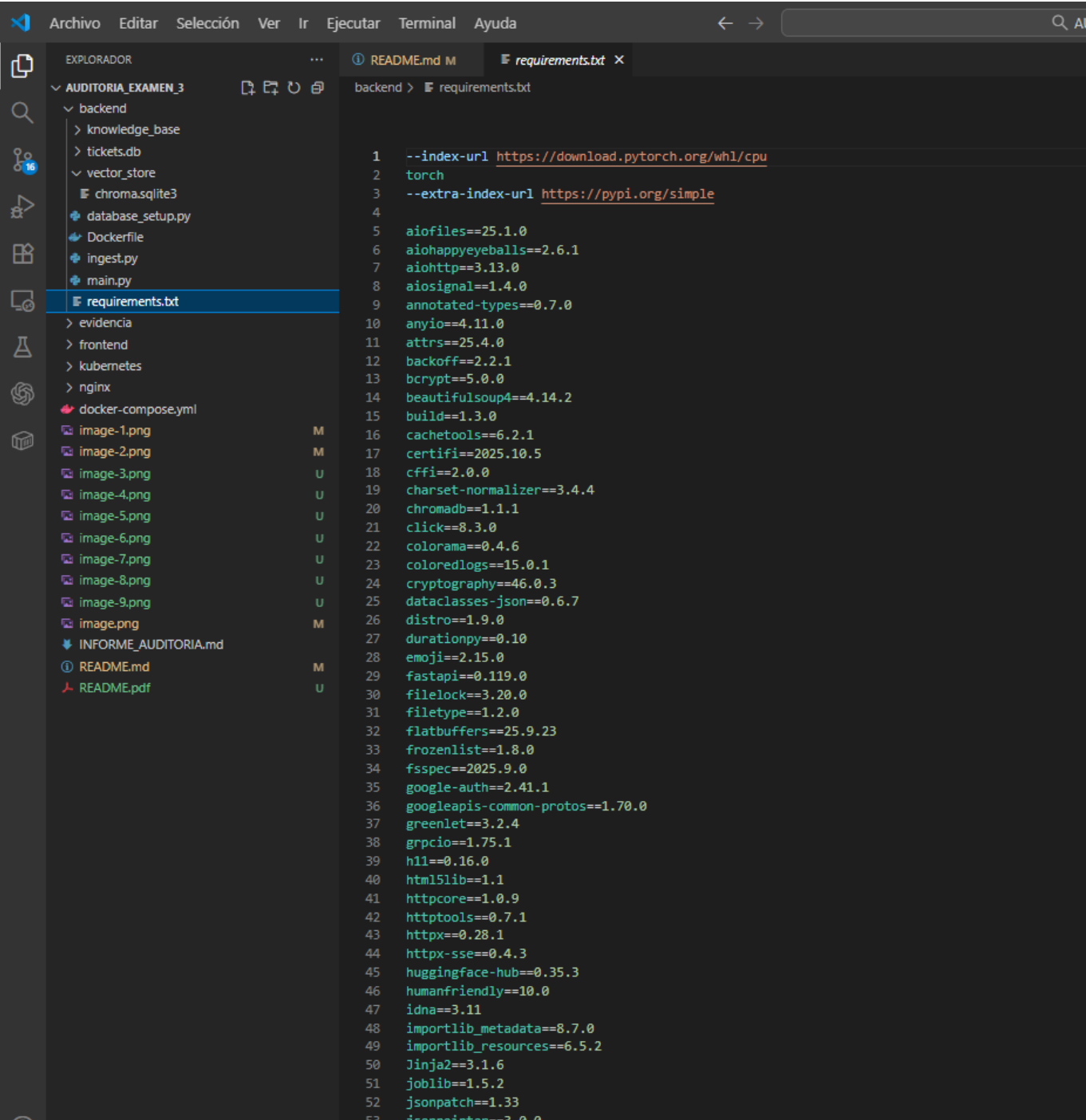
10. Requirements.txt

Descripción: Archivo requirements.txt mostrando dependencias con versiones desactualizadas y vulnerables.



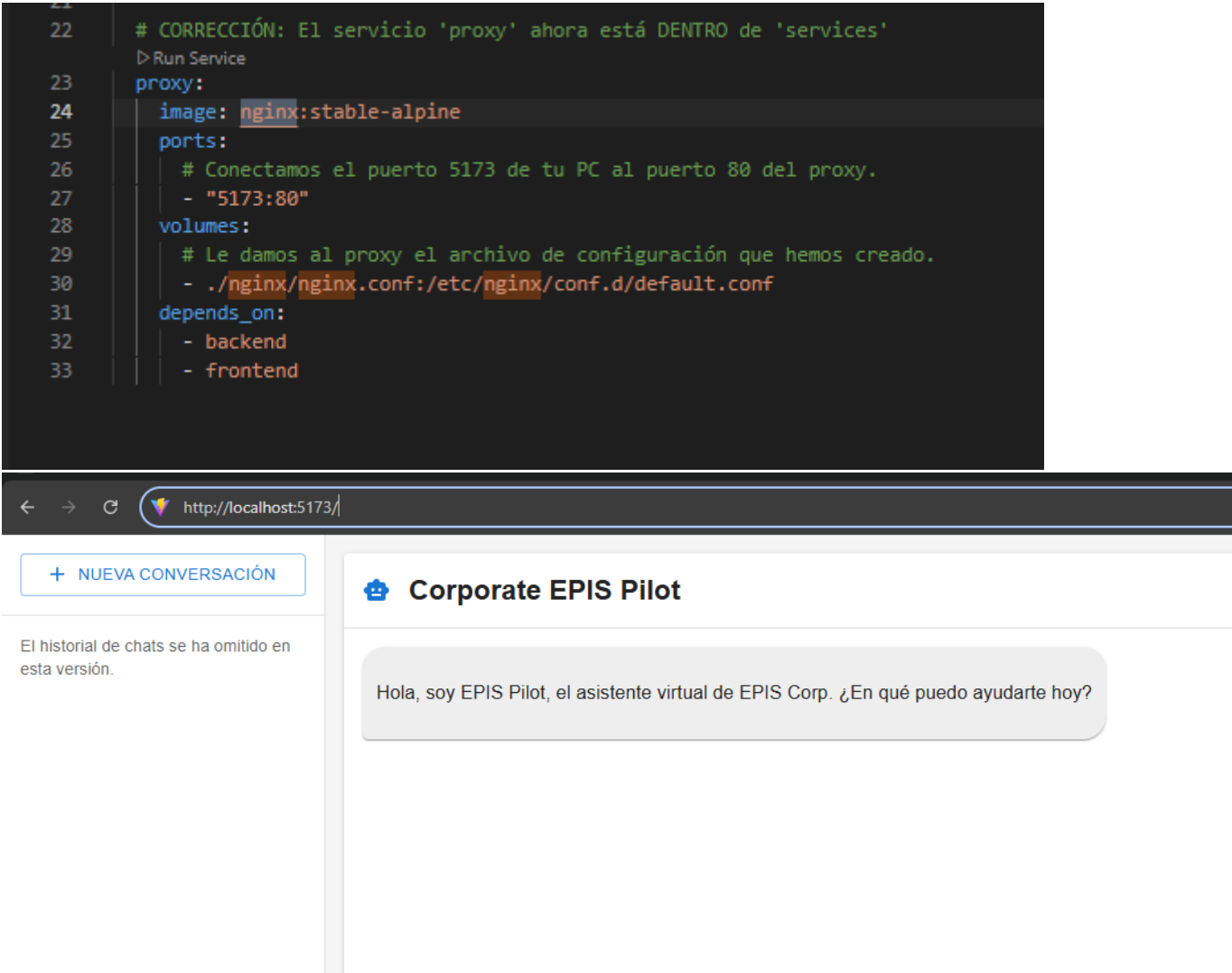
11. Base de Datos SQLite

Descripción: Visualización de la base de datos SQLite sin encriptación conteniendo tickets de soporte.



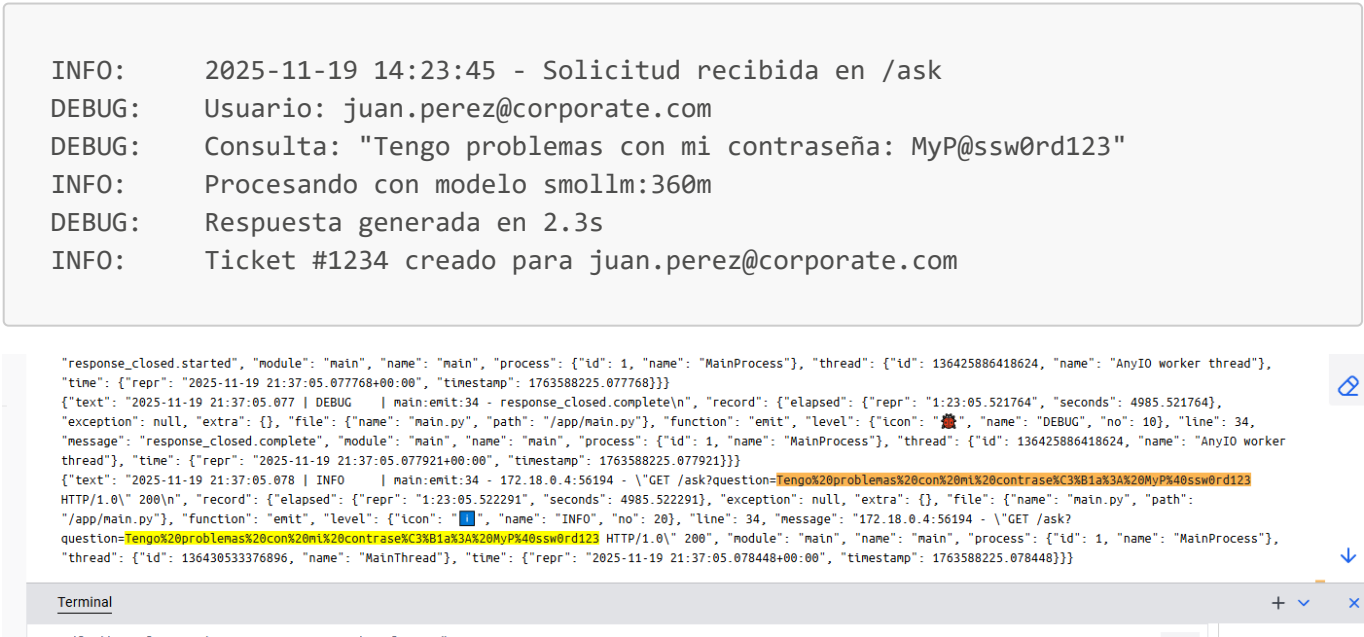
12. Configuración Nginx

Descripción: Configuración de nginx mostrando ausencia de SSL/TLS para HTTPS.



REGISTROS DE LOGS

Ejemplo de Log con Datos Sensibles Expuestos



POLÍTICAS INTERNAS REVISADAS

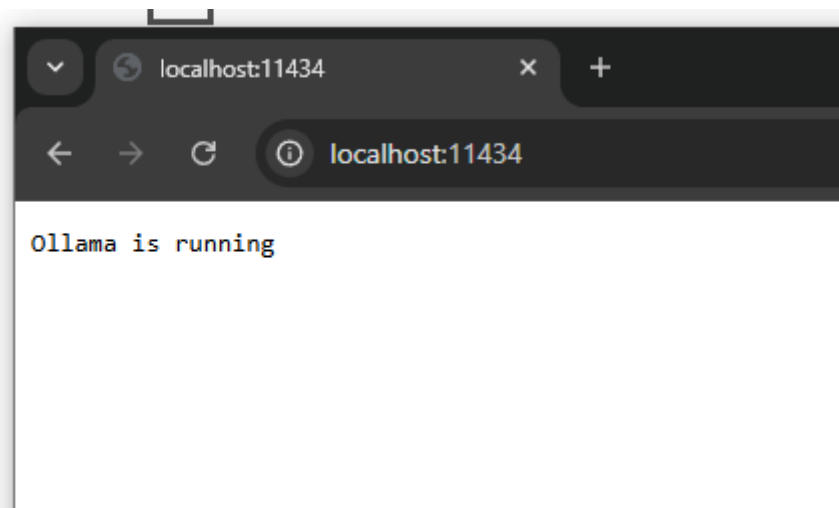
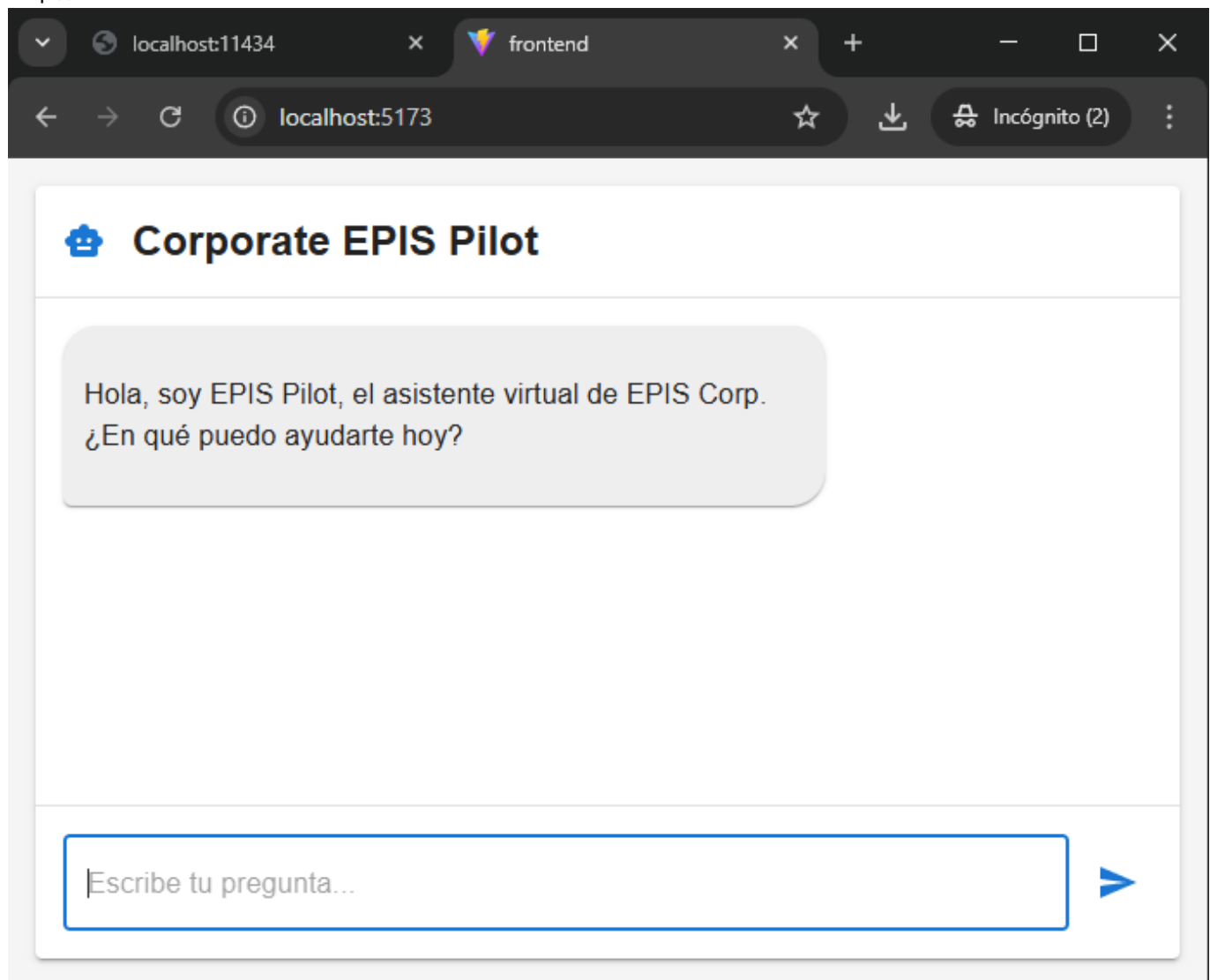
Política	Estado de Cumplimiento	Observaciones
Política de Control de Accesos	✗ No Cumple	No se implementan controles de autenticación
Política de Encriptación de Datos	✗ No Cumple	Sin HTTPS ni encriptación en base de datos
Política de Gestión de Logs	✗ No Cumple	Logs exponen información sensible sin sanitización
Política de Actualización de Software	✗ No Cumple	Dependencias desactualizadas sin proceso de gestión
Política de Segmentación de Red	⚠ Cumple Parcialmente	Redes Docker sin segmentación adecuada

ARCHIVOS DE CONFIGURACIÓN ANALIZADOS

docker-compose.yml

- Servicios: backend, frontend(vite), ollama
- Puertos expuestos:

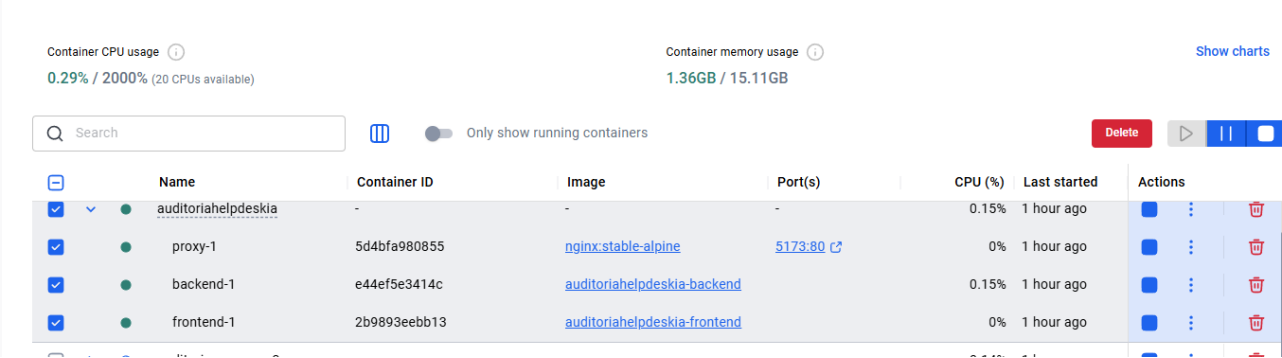
http://localhost:5173



http://localhost:11434

- Volúmenes persistentes configurados

- **Hallazgo:** Falta de health checks y restricciones de red



backend/main.py

- Framework: FastAPI
- **Hallazgo:** CORS inseguro (línea 47)
- **Hallazgo:** Endpoint sin autenticación (línea 111)
- **Hallazgo:** Logging sin sanitización (línea 141)

requirements.txt

```
56 kubernetes==34.1.0
57 langchain==0.3.27
58 langchain-chroma==0.2.6
59 langchain-community==0.3.31
60 langchain-core==0.3.79
61 langchain-huggingface==0.3.1
62 langchain-ollama==0.3.10
63 langchain-text-splitters==0.3.11
64 langdetect==1.0.9
```

- **Hallazgo:** Versiones vulnerables de langchain
- **Hallazgo:** Falta proceso de actualización automatizada